

Betriebssysteme

1 – ÜB – Einführung

Aufgabe 1: Grundkonzepte

- a) Welchen Nutzen hat ein Betriebssystem für den Endanwender eines Systems?
- b) Nach der Vorlesung sind zwei Grundzwecke des Betriebssystems Abstraktion von der realen, komplexen Hardware und Ressourcenverwaltung. Geben Sie für diese beiden Punkte jeweils ein weiteres, anderes Beispiel an als auf der Folie genannt!
- c) Wenn Ihr Smartphone im Stapelbetrieb laufen würde: Was würde beim Versuch passieren, eine Navigationsapp laufen zu lassen und parallel Musik zu hören?
- d) Arbeitet ein normaler Endanwender-PC in Echtzeitbetrieb? Geben Sie eine kurze Argumentation an, woran Sie Ihre Antwort festmachen.
- e) Im Zusammenhang mit Betriebssystemen ist oft die Rede von einem *Kern (kernel)*. Was versteht man unter einem Kern?
- f) Kann ein Betriebssystem zugleich im Mehrbenutzer-, aber nicht im Mehrprogramm-Betrieb arbeiten? Falls ja, wie? Falls nein, warum nicht?

Aufgabe 2: Betriebssysteme als Programme

- a) Ordnen Sie die folgenden Begriffe in die Ebenen des Aufbaus allgemeiner Rechner-systeme aus der Vorlesung ein. Recherchieren Sie ggfs. Begriffe, mit denen Sie nicht vertraut sind, vorher selbstständig. Geben Sie je eine kurze Begründung an!

Powershell, Ubuntu, Teams, Translation Lookaside Buffer, Windows Explorer

- b) Beschreiben Sie kurz, welchen Nutzen ein Systemaufruf open (Datei öffnen) unter den beiden Gesichtspunkten Abstraktion und Ressourcenverwaltung erfüllt.
- c) Beantworten Sie die folgenden Fragen zum Schaubild auf Folie 27:
 - Was ist ein Register?
 - Was ist der Stapel?
 - Welche Aufgabe hat der Dispatcher?
 - Warum ist dieses umständlich anmutende Vorgehen sinnvoll? Warum liest die Bibliotheksfunktion nicht selbst von der Festplatte?
 - Wann wird vom Benutzer- in den Kernmodus gewechselt und umgekehrt?
 - Wie kommt das Ergebnis vom read() zurück zur Anwendung?

d) Wie kann man die Systemaufrufe

- ... `open()`
- ... `fork()`
- ... `mkdir()`
- ... `mount()`
- ... `time()`

in einer Programmiersprache Ihrer Wahl einsetzen?

Werden die Systemaufrufe direkt getätigt oder liegt zwischen Systemaufruf und Ihrer Anwendung in der gewählten Sprache noch etwas anderes?

e) Schreiben Sie eine Anwendung in einer Sprache Ihrer Wahl, die mit Systemaufrufen ein Verzeichnis "mittagessen" erzeugt und darin eine Datei "ramen.txt" mit dem Inhalt "noodles meat shrooms" anlegt.

f) Schreiben Sie eine Anwendung in einer Sprache Ihrer Wahl, welche das Programm aus dem vorherigen Teilpunkt startet, wartet, bis das Programm beendet ist, und sich anschließend selbst beendet. Nutzen Sie auch hier Systemaufrufe.

Hinweis: Das Interpretieren von neuem Code bei Python ist kein Systemaufruf, der korrekte Systemaufruf wäre `es, py.exe` oder das Unix-Äquivalent `neu` zu starten.

g) In einer monolithischen Betriebssystem-Struktur läuft dieses als ein einzelnes, großes Programm, das an jeder Stelle Zugriff auf alle verfügbaren Systemaufrufe und Funktionalitäten hat. Argumentieren Sie kurz, weshalb dieser Ansatz ein hohes Fehlerpotenzial darstellt!

h) Wie hängt die Client/Server-Architektur für Betriebssysteme mit der verteilten Architektur zusammen?

i) Ein Kollege argumentiert: *"Bei der Client/Server-Architektur kann ja wieder jede Software-Komponente mit jeder anderen Komponente reden. Das widerspricht ja dem Gedanken der Schichtenarchitektur, durch Schichtung Sicherheit zu erzeugen, ist also eher ein Rückschritt."* Hat der Kollege Recht? Argumentieren Sie!